

Programmation Orientée Objet pour le Calcul Scientifique : Part II Deep Water

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- Outils et Langage (Tool)
- Gestion de la mémoire (Mem)
- **Orienté objet (OO)**
- Programmation statique (Static)

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 18 : la règle des exceptions

- Quelle approche utiliser pour gérer un comportement non souhaité/d'erreur ?
- Un code de retour ?

```
int func(int size) {  
    auto max_element = 1e3;  
    if (size > max_element) return 1;  
    else {  
        // do treatment  
        return 0;  
    }  
}  
  
int main() {  
    func(10000); // error code not tested...  
}
```

Item 18 : la règle des exceptions

- Problèmes des codes de retour :
 - ne sont (souvent) pas testés à l'appel de la fonction
 - polluent la signature de votre méthode (retour ou argument)
 - ne sont pas normalisés, votre convention devra être explicitée...
- En C++ et dans de nombreux langages objets, les **exceptions** viennent à la rescousse.
 - Elles offrent un mécanisme d'interruption en cas d'erreur par l'envoi/réception d'objet, par exemple de type `std::exception` ou dérivés en C++.

Item 18 : la règle des exceptions

- Un premier exemple

```
#include <iostream>
#include <exception>

void func(int size) {
    auto max_element = 1e3;
    if (size > max_element) throw std::exception();
    else {} // do treatment
}

int main() {
    try {
        func(10000); // error code not tested...
    }
    catch (std::exception& e) {
        std::cout << e.what() << std::endl;
    }
}
```

Item 18 : la règle des exceptions

- Une fonction/méthode pouvant lever une erreur est mise dans un bloc `try`
- L'exception est levée grâce à la méthode `throw`
- L'exception levée peut-être une classe du standard mais aussi une classe utilisateur
- le standard propose et utilise de nombreux types d'exceptions dérivés de la classe de base `std::exception`
- si l'exception est levée, l'exécution s'arrête et reprend dans le bloc `catch`
- l'absence de levée d'exception dans une méthode peut s'indiquer avec `noexcept`
- la libération propre de la mémoire dans le bloc `try` est gérée automatiquement (*stack unwinding*).

Item 18 : la règle des exceptions

- L'exception levée peut-être une classe du standard mais aussi une classe utilisateur.

```
struct exception {  
    std::string message;  
};  
  
int main(){  
    try {  
        throw exception{"an error occurred"};  
    }  
    catch (exception& e) {  
        std::cout << e.message << std::endl;  
    }  
}
```

Item 18 : la règle des exceptions

- le standard propose et utilise de nombreux types d'exceptions dérivées de la classe de base `std::exception`, par exemple :
 - `std::bad_alloc` : levée lors d'un échec de `new`
 - `std::bad_cast` : levée lors d'un échec de `dynamic_cast`
 - `std::bad_typeid` : levée lors d'un échec de `typeid`
 - `std::io_base::failure` : levée lors d'un échec des fonctions de `iostream`
 - `std::bad_weak_ptr` : levée lorsqu'un `weak_ptr` pointe sur un objet détruit

le catalogue est disponible : <https://en.cppreference.com/w/cpp/error/exception>

le type de l'exception donne des informations sur la nature de l'erreur.

Il faut inclure le header : `#include <exception>`

Item 18 : la règle des exceptions

- Toutes ces exceptions sont à votre disposition pour votre code. Ainsi vous pourrez utiliser :
 - `std::runtime_error` pour une erreur à l'exécution
 - `std::range_error` pour un problème de taille de tableau
 - `std::invalid_argument` pour un argument incorrect dans une fonction
- la méthode `what()` de ces exceptions vous retournera une chaîne de caractère indiquant son type.
- Ces exceptions peuvent prendre en argument une chaîne `char*`

```
throw std::runtime_error{"My error message"};
```

Item 18 : la règle des exceptions

- Toutes ces exceptions sont à votre disposition pour votre code.
- la méthode `what()` de ces exceptions vous retournera une chaîne de caractère indiquant son type.
- Ces exceptions peuvent prendre en argument une chaîne `char*`

```
throw std::runtime_error{"My error message"};
```

Exo 18.1 : écrivez une fonction qui affiche l'entier qui lui est passé en argument s'il est positif et qui lève une exception de type `std::invalid_argument` s'il est négatif. Un message explicatif sera passé à l'exception. Appelez cette fonction depuis un `main` avec un entier positif et un entier négatif.

Item 18 : la règle des exceptions

- si l'exception est levée, l'exécution s'arrête et reprend dans le bloc `catch` 1/3
 - il faut préciser le type de l'exception en argument

```
catch(std::runtime_exception& e ) { ...}
```

- un bloc de `catch` peut donc être écrit pour chaque type d'exception
 - cela permet de segmenter le traitement en fonction de l'exception levée
- l'appel de `catch(...)` capturera tous les types d'exception.
 - Cela perd de l'intérêt puisque l'on ne sait pas quelle est l'exception concernée
 - Peut se mettre en dernier rempart, si on veut à tout prix que le code n'échoue pas...

Item 18 : la règle des exceptions

- si l'exception est levée, l'exécution s'arrête et reprend dans le bloc `catch` 2/3

```
struct exception {std::string message;};  
void func(int size) {  
    auto max_element = 1e3;  
    if (size > max_element) throw std::invalid_argument("i must be < 1000");  
    else { throw exception{"an error occured"}; } // error in treatment  
}  
int main(){  
    try {  
        func(5e2); // first catch  
        func(5e3); // second catch  
    }  
    catch (exception& e) { std::cout << e.message << std::endl; }  
    catch (std::invalid_argument& e) {  
        std::cout << " invalid argument : " << e.what() << std::endl; }  
}
```

Item 18 : la règle des exceptions

- l'appel de `catch(...)` capturera tous les types d'exception. 3/3

```
struct exception {std::string message;};
void func(int size) {
    auto max_element = 1e3;
    if (size > max_element) throw std::invalid_argument("i must be < 1000");
    else { throw exception{"an error occured"}; } // error in treatment
}
int main(){
    try {
        func(5e2);
        func(5e3); // both caught in the same block
    }
    catch (...) {
        std::cout << "An exception occured" << std::endl;
    }
}
```

Item 18 : la règle des exceptions

- si l'exception est levée, l'exécution s'arrête et reprend dans le bloc `catch`
 - il faut préciser le type de l'exception en argument

```
catch(std::runtime_exception& e ) { ...}
```

- un bloc de `catch` peut donc être écrit pour chaque type d'exception
- l'appel de `catch(...)` capturera tous les types d'exception.

Exo 18.2 : reprenez votre exercice 18.1 et ajoutez un bloc `try/catch` pour capturer votre exception et en afficher le message.

Item 18 : la règle des exception

- la libération propre de la mémoire dans le bloc `try` est gérée automatiquement
 - *stack unwinding*

```
struct A {  
    ~A() {std::cout << "~A()" << std::endl;}  
};  
int main() {  
    try {  
        A a{};  
        throw std::exception{};  
    }  
    catch(...) {}  
} // ~A() is called. Ressource properly destroyed.
```

Item 18 : la règle des exception

- la libération propre de la mémoire dans le bloc try est gérée automatiquement
 - *stack unwinding*
- cette gestion de la mémoire a un coût :
 - peut empêcher des optimisations du compilateur
- en raison de cette gestion mémoire liée à l'exception :
 - il n'est pas possible de lever une exception dans des destructeurs

```
struct A { ~A() {throw std::runtime_error("failed");} };  
  
int main() {  
    try { A a{}; }  
    catch(...) {}  
} // crash: terminate called after throwing an instance of 'std::runtime_error'
```

Item 18 : la règle des exception

- l'absence de levée d'exception dans une méthode peut s'indiquer avec `noexcept`

```
void no_exception_function noexcept {  
    // function not throwing any exception  
    // and calling only functions not throwing any exception  
}
```

- Cette qualification n'est **pas vérifiée** à la **compilation** mais à l'**exécution**
- L'exécution d'une fonction `noexcept` provoque un plantage

```
void function() noexcept {throw std::runtime_error("failed");}  
int main() {  
    try { function(); }  
    catch(...) {}  
} // crash: terminate called after throwing an instance of 'std::runtime_error'
```

Item 18 : la règle des exception

- les destructeurs sont par défaut `noexcept`
- pour aller plus loin : `noexcept` peut être conditionnel
 - pour exprimer `noexcept` si et seulement si une autre fonction l'est...

Exo 18.3 : reprenez votre exercice 18.2 et créez maintenant votre propre classe `exception`, qui vous permettra par exemple de stocker la valeur de l'argument d'entrée de votre fonction qui a provoqué son déclenchement. Ecrivez le bloc `catch` pour la capturer. Au sein du bloc `catch` vous afficherez la valeur de l'entier stocké. Finalement constatez ce qui se passe lorsque vous mettez votre fonction d'affichage en `noexcept`.

Item 19 : Interfacer sans interface

Qu'est-ce qu'une *interface* en programmation objet ?

- La définition d'un contrat
- un unique code pour manipuler différente implémentation de ce contrat

En java par exemple

```
public interface Solver {  
    public void solve();  
}  
public class DirectSolver : implements Solver {  
    public void solve() { System.out.println("Direct Solver");}  
}  
public class IterativeSolver : implements Solver {  
    public void solve() { System.out.println("Iterative Solver");}  
}
```

Item 19 : Interfacer sans interface

Qu'est-ce qu'une *interface* en programmation objet ?

- Une interface est une classe mère qui oblige ses enfants à implémenter ses méthodes.
- Les méthodes d'une interface sont la plupart du temps vides.

En C++ ?

- C++ ne contient pas de type `interface`
- Il faut utiliser la notion de *méthode virtuelle pure*

Item 19 : Interfacer sans interface

Méthode virtuelle pure 1/5

- la syntaxe est :

```
class MyInterface {  
public:  
    virtual void my_pure_virtual_method = 0;  
    virtual ~MyInterface() = default;  
}
```

- une méthode virtuelle pure n'est habituellement **pas définie** (juste déclarée)
- une classe contenant une méthode virtuelle pure **ne peut pas s'instancier**
- une méthode virtuelle pure doit être **surchargée** dans **toutes les classes enfant**

Item 19 : Interfacer sans interface

Méthode virtuelle pure 2/5

- une méthode virtuelle pure n'est habituellement **pas définie** (juste déclarée)
 - une méthode pure n'a pas en soi de comportement
 - néanmoins les compilateurs le permettent,
 - il faut alors séparer définition et déclaration
 - peut servir pour un comportement par défaut commun à tous les enfants

```
class MyInterface {  
public :  
    virtual void function() = 0;  
    virtual ~MyInterface() = default;  
};  
  
void MyInterface::function() {std::cout << "Interface::function";}
```

Item 19 : Interfacer sans interface

Méthode virtuelle pure 3/5

- une classe contenant une méthode virtuelle pure **ne peut pas s'instancier**
 - elle ne fait aucune action ;
 - elle ne sert qu'à manipuler génériquement ses enfants.

```
class MyInterface {  
public :  
    virtual void function() = 0;  
    virtual ~MyInterface() = default;  
};  
int main() {  
    MyInterface i{};  
    // ERROR: cannot declare variable 'i' to be of abstract type 'MyInterface'  
}
```

Item 19 : Interfacer sans interface

Méthode virtuelle pure 4/5

- une méthode virtuelle pure doit être **surchargée** dans **toutes les classes enfant**
 - un enfant ne surchargeant pas la(les) méthode(s) virtuelle(s) pure(s) de son parent ne peut s'instancier

```
class MyInterface {  
public :  
    virtual void function() = 0;  
    virtual ~MyInterface() = default;  
};  
class MyImplementation : public MyInterface {};  
  
int main() {  
    MyImplementation mip{};  
    // ERROR: cannot declare variable 'mip' to be of abstract type 'MyImplementation'  
}
```

Item 19 : Interfacer sans interface

Méthode virtuelle pure 5/5

- L'intérêt est donc d'avoir de l'héritage et une manipulation polymorphique

```
class ISolver {  
public :  
    virtual void solve() = 0;  
    virtual ~ISolver() = default;  
};  
class MySolver : public ISolver {  
public:  
    void solve() override {std::cout << "MySolver::solve()" << std::endl;}  
};  
class MyOtherSolver : public ISolver {  
public:  
    void solve() override {std::cout << "MyOtherSolver::solve()" << std::endl;}  
};
```

Item 19 : Interfacer sans interface

Création d'une interface 1/2

- On crée un **pointeur d'interface** pour manipuler l'implémentation
 - à l'ancienne (raw pointer)

```
int main() {  
    ISolver* solver = new MySolver{};  
    solver->solve();  
    delete solver; // MySolver is destroyed  
  
    solver = new MyOtherSolver{};  
    solver->solve();  
    delete solver; // MyOtherSolver is destroyed  
}
```

Item 19 : Interfacer sans interface

Création d'une interface 2/2

- On crée un **pointeur d'interface** pour manipuler l'implémentation
 - avec du C++ moderne (smart pointer)

```
int main() {  
    using ISolverPtr = std::unique_ptr<ISolver>;  
    ISolverPtr solver = std::make_unique<MySolver>();  
    solver->solve();  
  
    solver = std::make_unique<MyOtherSolver>(); // MySolver is destroyed  
    solver->solve();  
} // MyOtherSolver is destroyed
```

Item 19 : Interfacer sans interface

- L'interface est donc du polymorphisme sous contrainte :
 - il est obligatoire de surcharger dans les enfants
- Cela permet d'imposer un contrat, par exemple le contrat `ISolver`

Exo 19.1 : créez une classe `Interface` avec une méthode `compute` et créez deux classes dérivées `Implem1` et `Implem2` qui héritent d' `Interface` . `Interface` doit obliger ses classes dérivées à définir la méthode `compute` . Dans votre `main` , créez deux pointeurs d' `Interface` (avec les deux implémentations) et passez-les dans une fonction `apply(Interface*)` qui appelle la méthode `compute` .
Recommencez l'exercice avec des smart pointeurs (`shared_ptr` ou `unique_ptr`).

Item 19 : Interfacer sans interface

- **Exo 19.2** Créez une classe `Solvers` qui contient un tableau `std::vector` de solveurs sous la forme de pointeur (`std::unique_ptr`) d'interface `ISolver` , et deux classe filles `MySolver` et `MyOtherSolver` .
 - Cette classe `Solvers` contiendra :
 - une méthode `addSolver` qui prendra en argument le nom du solveur et comme argument template son type.
 - une méthode `solve` qui parcourera le tableau de solveur et appellera la méthode `solve` de chaque solveur au moyen d'un algorithme `std::for_each` et d'une λ -function.
 - Dans un programme principal :
 - ajoutez plusieurs solveurs dans cette classe
 - appelez la méthode `solve`